

PROJECT DOCUMENTATION

(BOB ROSS WITHOUT ROS)

1. Division I — Learning / Iteration Phase

Captures the full exploratory journey of the project, preserved for anyone who wants to understand how the team arrived at the final approach — not just what the final approach is.

1.1 Individual Iterations

The project followed an iterative development path in which each phase produced the knowledge and constraints that shaped the next. The stages below summarise how the team progressed from initial study to the completed arm, including the main alternatives that were tried and the reason behind each final choice.

1. **Study of forward and inverse kinematics.** At the start of the project, all team members studied forward kinematics (FK) and inverse kinematics (IK), and each proposed different methods for applying them to the arm.

Forward kinematics. Denavit–Hartenberg (D-H) transformation matrices were compared with an explicit trigonometric formulation. The trigonometric form was kept because D-H adds redundant matrix multiplications for a planar arm.

Inverse kinematics. Four methods were considered. Analytic geometric IK is fast but struggles with the redundancy of three joints reaching a two-dimensional (x, y) target unless arbitrary constraints are added. A numerical gradient-descent method is easy to implement but converges slowly, can settle in local minima near joint limits, and has a variable runtime. The basic Jacobian transpose is computationally simple but converges slowly and oscillates near singular poses. Damped Least Squares (DLS) handles singularities smoothly by damping the inversion, and was selected. A basic IK resource used during this study is available in the repository root as [GradientDescentBasedIK.pdf](#).

2. **Study of computer vision.** All team members then studied computer vision and its various techniques, and three image-processing pipelines were tried.

First pipeline. Grayscale conversion, Gaussian blur, bilateral filtering, Canny edge detection and morphological operations. It was fast, but produced double lines and fragmented paths and gave no shading.

Second pipeline. A bilateral filter with CLAHE and XDoG. It produced clean sketch-like binary lines, but without structured path ordering or tonal shading.

Third pipeline. The second pipeline extended with contour extraction and scanline hatching. It extracts smooth, continuous contours and generates structured hatching paths mapped to robot workspace coordinates, and was adopted.

3. **Simulation for the mid-project evaluation.** For the mid-project evaluation, the team built a simulation of the final arm that was to be constructed. The simulation brought the kinematics and vision work together with a trajectory generator.

Trajectory generation. Four velocity-profile approaches were compared. A linear profile has unbounded acceleration at the path boundaries, which causes mechanical jerk and motor stall. A cubic polynomial profile has continuous velocity but discontinuous acceleration, which causes structural vibration. A quintic polynomial profile has continuous velocity and acceleration, but its rigid multi-segment timing makes real-time speed adjustment difficult. The final choice was a bell-shaped speed profile with look-ahead braking and joint-limit bounding, which keeps the motion smooth while the Jacobian is used to hold joint velocity and acceleration within limits.

4. **Hardware development.** The team then began the hardware phase. This involved identifying the mechanical constraints of the arm and working through several phases of combined software–hardware debugging, along with related integration work. Alternatives were compared for each part of the hardware.

Joint actuators. NEMA 17 stepper motors gave open-loop positioning and added considerable weight to the cantilevered links, and standard RC servos had low torque and poor position feedback. Dynamixel servos gave precise feedback and high torque and can be daisy-chained, so they were selected.

Pen lift. A solenoid has a binary, slamming action that damaged the pen tip. A gear-and-slider mechanism let the arm sag under gravity and link deflection when fully extended, so the pen pressed harder than when the arm was folded. A string and micro-servo mechanism removed this dependence on arm reach and was selected.

Communication. A generic USB-to-RS485/TTL FTDI adapter suffered packet dropouts and timing latency at 1 Mbaud. An ESP32 with an RS485/TTL transceiver offers high performance for embedded standalone control but needs custom protocol bridging for live debugging from a PC. The official Robotis U2D2 converter gives direct, low-latency communication and was selected, with the ESP32 option retained only as a possible embedded standalone deployment path.

5. **Completion.** Using the final software and hardware stack documented in Section 1.2, the team completed the arm. The system can now reproduce images with a high degree of accuracy, as shown by the comparison images in Section 2.4.

1.2 Approach Documentation

Software Document

The final software approach selected for each module is listed below.

Module	Final approach	Justification
Forward kinematics (FK)	Explicit trigonometric formulation (FK in <code>Kinematics.py</code>): $x = L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2) + L_3 \cos(\theta_1 + \theta_2 + \theta_3)$ and $y = L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2) + L_3 \sin(\theta_1 + \theta_2 + \theta_3)$.	Direct calculation with low computational overhead, which suits real-time control loops.
Inverse kinematics (IK)	Damped Least Squares (<code>ik_fast_dls</code>), warm-started from the previous joint angles, with damping factor $\lambda = 0.05$.	Warm-starting favours nearby, continuous poses along a drawing stroke. Damping keeps the pseudo-inverse well behaved near singular poses, and the pseudo-inverse resolves the extra degree of freedom of three joints reaching a two-dimensional (x, y) target. Each solve returns a success flag.
Trajectory generation	Bell-shaped speed profile with look-ahead braking and Jacobian-based joint limits (<code>generate_continuous_trajectory</code>).	Task-space speed follows a smooth bell-shaped curve, is capped by a look-ahead braking curve near the end of each path, and is limited at every frame through the Jacobian so that joint velocity and acceleration stay within <code>MAX_JOINT_VEL</code> (4.0 rad/s) and <code>MAX_JOINT_ACC</code> (1.5 rad/s ²).
Image processing (CV)	Bilateral filter, CLAHE and XDoG, followed by Otsu thresholding and removal of small components. Outlines are extracted with RDP simplification and B-spline smoothing and combined with scanline hatching (<code>image_to_robot_paths</code>).	Produces smooth, continuous outline strokes together with structured scanline hatching paths for shaded regions, all mapped to robot workspace coordinates.

Hardware Document

The final hardware component selected for each function is listed below.

Component	Final selection	Justification
Controller computer	Raspberry Pi, running the Python control software and connected to the U2D2 and the Arduino Nano.	Hosts the control software on the arm itself, and is operated from a laptop over SSH through an Ethernet cable.
Joint actuators (joints 0, 1, 2)	Dynamixel servos: XM540 (shoulder, ID 0), XM430 (elbow, ID 1) and XM430 (wrist, ID 2), operated in Extended Position Control mode (multi-turn) at 1 Mbaud.	Precise position feedback, high torque, and a daisy-chainable bus.
Pen-lift mechanism (Z-axis)	String and micro-servo mechanism. The pen is raised and lowered by <code>pen_up()</code> and <code>pen_down()</code> , which send the bytes U and D over a serial link to an Arduino Nano that drives the micro servo.	The servo tension pulls the pen up off the paper, and releasing the tension lets gravity and the pen's compliance set the contact force. This decouples the Z-axis drive from link gravity deflection, so the pen contact force is the same whether the arm is fully extended or folded.
Communication	Official Robotis U2D2 converter, connected directly to the Dynamixel bus through the Dynamixel SDK at 1,000,000 bps using Protocol 2.0.	Direct, low-latency communication between the controlling computer and the Dynamixel bus.

2. Division II — Final Product

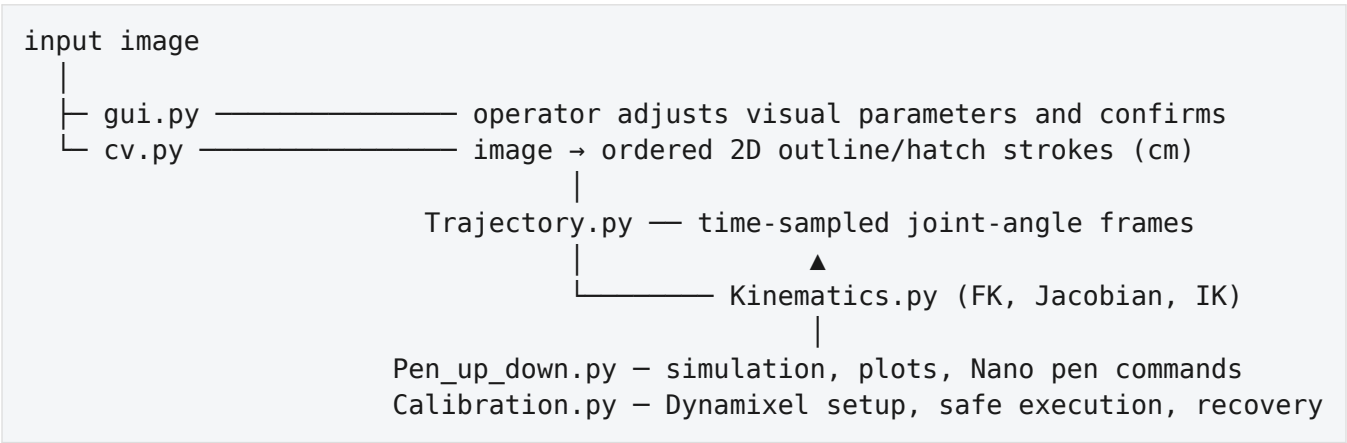
The polished, deployable outcome of the project.

2.1 Codebase

BobRoss Code Explainer

Scope

This document explains the seven Python modules in `BobRoss/` and how they collaborate to turn a raster image into motion of a three-link planar painting arm:



The integrated `BobRoss` path uses centimetres for Cartesian workspace coordinates, radians for joint angles, seconds for time, radians/second and radians/second² for joint limits, and Dynamixel encoder ticks at the final hardware boundary. The arm is planar: the task is pen-tip position (x, y) . Three joint variables control two task variables,

leaving one redundant degree of freedom; the project resolves this numerically with warm-started damped least-squares inverse kinematics.

Operational distinction: `Calibration.py` is executable hardware-control code, not a safe library import. Its top-level statements open a Dynamixel port, prompt for zero calibration, enable torque, and run an input loop.

Dependency map

File	Main responsibility	Project dependencies	Main consumers
<code>Configurations.py</code>	Shared physical model and tuning constants	None	All modular files
<code>Kinematics.py</code>	Arm geometry, Jacobian, bounded inverse kinematics	<code>Configurations.py</code>	Planner, simulator, hardware runner
<code>Trajectory.py</code>	Convert polylines to smooth, limited joint frames	Configuration + kinematics	Simulator and hardware runner
<code>cv.py</code>	Convert image pixels to physical drawing paths	Configuration	Both GUIs; hardware imports it
<code>gui.py</code>	Lightweight image/path approval UI	CV pipeline	<code>Calibration.py</code> command t
<code>Pen_up_down.py</code>	Full simulator and Nano serial pen interface	Configuration, CV, kinematics, planner	<code>Calibration.py</code> imports the pen controller
<code>Calibration.py</code>	Calibrate and command Dynamixels; log/recover drawing	All operational modules	Directly executed program

1. `Configurations.py` — the project-wide physical and algorithmic contract

Why the file is needed

This module is the single source of truth for values which must agree across subsystems. If the CV mapper uses one canvas centre while kinematics assumes a different arm geometry, visually correct strokes land in the wrong location. If the planner samples at a rate different from the hardware loop, its numerical velocity limits no longer describe real movement. Collecting constants here makes the system model inspectable and reduces duplicated “magic numbers.”

It defines no functions or classes and has no side effects. Importing it only creates named constants. It imports NumPy for the commented degree-based limit alternative; the active limits are numeric radian literals.

Link geometry and reach

Symbol	Current value	Meaning	Used by
<code>L1</code>	<code>15.9 cm</code>	Base/shoulder to elbow link length	FK, Jacobian, IK
<code>L2</code>	<code>15.05 cm</code>	Elbow to wrist link length	FK, Jacobian, IK
<code>L3</code>	<code>11.1 cm</code>	Wrist to pen-tip link length	FK, Jacobian, IK
<code>L_TUPLE</code>	<code>(L1, L2, L3)</code>	Default link vector passed to math functions	<code>Kinematics.py</code>
<code>MAX_REACH</code>	<code>L1 + L2 + L3</code>	Outer radial reach bound	<code>ik_fast_dls()</code>

For angles $q = [q_1, q_2, q_3]$, the three physical link headings are q_1 , $q_1 + q_2$, and $q_1 + q_2 + q_3$. These values must be measured from the actual joint axes to the pen tip—not merely the visible arm segments—because the same geometry drives planning, simulation, and reachability checks.

Joint, motion, and timing constants

`JOINT_LIMITS` provides one lower/upper radian interval for shoulder, elbow, and wrist.

`Kinematics.clip_to_joint_limits()` applies them at every IK iteration; the integrated simulator applies them again after trajectory smoothing. These are angle-model limits, distinct from `Calibration.py`'s measured servo-tick limits.

Constant	Current value	Why it matters
<code>MAX_LINEAR_SPEED</code>	30.0 cm/s	Nominal Cartesian speed requested for drawing.
<code>MAX_JOINT_VEL</code>	4.0 rad/s	Per-joint velocity ceiling transformed to a Cartesian bound by the Jacobian.
<code>MAX_JOINT_ACC</code>	1.5 rad/s ²	Per-joint acceleration ceiling used between planner frames.
<code>FPS</code>	30	Intended trajectory sample/execution frequency.
<code>DT</code>	1/FPS	Time interval used in integration, velocity estimation, sleeping, and animation.
<code>REVIEW_WINDOW</code>	5.0 s	Simulator velocity-history window.

The important numerical relation is $dq \approx (q_{\text{next}} - q_{\text{current}}) / DT$. Altering `FPS` changes not only display smoothness, but the planner's distance step and the real runner's `time.sleep(1/FPS)` delay.

Inverse-kinematics tuning

`IK_MAX_ITER=100` limits the number of DLS corrections; `IK_TOL=1e-2` is the Cartesian convergence tolerance in centimetres; `IK_DAMPING=0.05` regularises the pseudo-inverse; and `IK_MAX_STEP=0.30 rad` limits the norm of one correction. These values are read by `Kinematics.ik_fast_dls()`. More damping/less step size tends to be safer but slower; less damping risks very large motion near singular configurations.

Trajectory shaping

`SAFE_DECEL_RATE=7.0 cm/s2` supports the braking rule in `Trajectory._desired_speed()`.

`MIN_SPEED_FLOOR`, `NEAR_TARGET_DIST`, `NEAR_TARGET_SPEED_FLOOR`, and `FAR_TARGET_SPEED_FLOOR_FRAC` prevent numerical progress from stalling while allowing a slow final approach. `TRAVEL_SPEED_MULT=1.5` makes pen-up simulator travel faster than drawing; `PAUSE_DURATION=0.2 s` becomes a number of dwell frames around pen transitions.

Smoothing, vision, and mapping

`SG_WINDOW_LENGTH=15` and `SG_POLYORDER=3` configure Savitzky-Golay smoothing in `Pen_up_down.py`. The XDoG defaults (`XDOG_SIGMA`, `XDOG_K_SIGMA`, `XDOG_EPSILON`, `XDOG_PHI`, `XDOG_GAMMA`) seed the integrated GUI sliders. Contour constants set minimum contour length/point count and RDP/B-spline aggressiveness.

`XDOG_AUTO_TUNE=True` is currently defined but not read by any module; automatic parameter estimation still happens only when a CV function receives `None` parameters.

`TARGET_CANVAS_W=23.0` and `TARGET_CANVAS_H=23.0` define the maximum drawing rectangle in centimetres. `CANVAS_CENTER_X=26.5` and `CANVAS_CENTER_Y=0.0` position it in the arm frame.

`cv.map_paths_to_workspace()` uses these values to map an image centre into a physical centre while retaining aspect ratio.

2. Kinematics.py — forward geometry, differential motion, and DLS IK

Purpose and use in the project

This module converts between joint space and planar pen-tip space. `Trajectory.py` uses its Jacobian and IK on every planned frame; `Pen_up_down.py` uses FK and link positions for simulation; `Calibration.py` uses IK for direct Cartesian moves and receives trajectory results generated through it. Without it, an image path would remain a list of (x, y) points with no feasible motor angles.

The task has two outputs (x, y) but three angles, so its Jacobian is 2×3 and cannot be ordinarily inverted. The code therefore uses an iterative damped pseudo-inverse. Passing the last solution as `q_init` is deliberate: it favors nearby, continuous poses along a drawing stroke.

`FK(q, L=L_TUPLE)`

What it does: Returns the pen-tip Cartesian coordinate (x, y) from the three joint angles.

`Pen_up_down.generate_full_trajectory()` calls it to find the current tip before a pen-up travel segment; IK calls it each iteration to measure current error.

Mathematics:

$$x = L_1 \cos q_1 + L_2 \cos(q_1 + q_2) + L_3 \cos(q_1 + q_2 + q_3)$$

$$y = L_1 \sin q_1 + L_2 \sin(q_1 + q_2) + L_3 \sin(q_1 + q_2 + q_3)$$

It returns a two-item tuple rather than an array. The optional `L` makes geometry experiments possible, but normal callers use shared configuration geometry.

`arm_link_positions(q, L=L_TUPLE)`

What it does: Computes and returns $(p0, p1, p2, p3)$: base, elbow, wrist, and pen tip. It uses the same cumulative-angle geometry as FK, but retains intermediate points. `Planar3D0FSimApp.draw_arm()` uses these points to render the two-link arm portion and pen/brush segment. It does not solve IK or command any hardware.

`compute_jacobian(q, L=L_TUPLE)`

What it does: Builds the analytical 2×3 positional Jacobian J . It is used in DLS IK and in trajectory speed limiting.

The differential relationship is:

$$[\dot{x}, \dot{y}]^T = J(q)[\dot{q}_1, \dot{q}_2, \dot{q}_3]^T.$$

Each column is the partial derivative of pen position with respect to one joint. Joint one affects every link; joint three affects only the final link. For example, its first x derivative is $-L_1 \sin(q_1) - L_2 \sin(q_1+q_2) - L_3 \sin(q_1+q_2+q_3)$. In stretched or otherwise singular poses, the columns do not provide enough independent Cartesian control, which motivates damping.

`damped_pseudo_inverse(J, damping=IK_DAMPING)`

What it does: Returns the regularised pseudo-inverse:

$$J_{\lambda}^+ = J^T(JJ^T + \lambda^2 I)^{-1}.$$

The addition of $\lambda^2 I$ makes the 2×2 task-space matrix better conditioned near singularity. The implementation uses `np.linalg.solve` rather than explicitly calculating an inverse. `ik_fast_dls()` uses $J^+ e$ as a joint correction; `Trajectory.py` uses $J^+ \hat{u}$ as joint velocity required per unit Cartesian speed.

`clip_to_joint_limits(q)`

What it does: Copies the input angle vector and clips each angle to the corresponding configured lower/upper bound. It is invoked after each IK update so numerical correction cannot retain a forbidden model pose. The simulator imports it but its smoothing code performs an equivalent direct `np.clip` instead.

`ik_fast_dls(target_x, target_y, q_init, L=L_TUPLE, max_iter=IK_MAX_ITER, tol=IK_TOL)`

What it does: Iteratively seeks a limited three-angle pose that reaches a target coordinate. It returns `(q, success, third_value)`: when successful, `third_value` is the number of iterations used; when unsuccessful, it is `max_iter`. It is not an error norm.

Step-by-step behavior:

1. It measures target radius with `hypot`. A point outside `MAX_REACH` is scaled inward to `MAX_REACH - 0.01`, avoiding an obvious outer-reach failure.
2. It copies `q_init` as the current pose. Warm-starting from the prior frame is the main continuity mechanism for a redundant arm.
3. Each iteration calls FK, builds position error `e = [target_x - curr_x, target_y - curr_y]`, and succeeds when `||e|| < tol`.
4. It calculates `dq = J+e` with the damped pseudo-inverse.
5. A negligible update norm (`< 1e-4`) stops iteration; an oversized update is scaled to `IK_MAX_STEP`.
6. It adds the update, wraps angles into `[-π, π)`, then clips them to `JOINT_LIMITS`.
7. It returns the final pose even on failure.

Significant implication: `Calibration.go_to_xy_3dof()` only prints a warning when `success` is false, and `Trajectory.py` ignores this flag. A failed result can still become a motor command or later frame. Target radius clipping also cannot account for internal holes in workspace or configured joint limits.

3. `Trajectory.py` — arc-length path following under joint limits

Purpose and use in the project

This module converts a polyline of physical `(x, y)` points into joint-angle vectors, one per DT. It is called by `Calibration.follow_path()` for real pen-down execution and by `Planar3DOFSimApp.generate_full_trajectory()` for both pen-up travel and pen-down drawing. Its output is position frames; the controller sends them at FPS, and the simulator derives display velocities with `np.gradient`.

It is needed because repeated waypoint IK alone gives no controlled timing, no smooth acceleration behavior, and no guarantee that a small Cartesian step will not demand excessive joint motion. This module uses arc length rather than original image point spacing, then constrains progress using both task-space and joint-space reasoning.

`get_interpolated_point(points, cum_dist, current_segment, target_dist)`

The planner stores cumulative arc length at each segment boundary. This helper locates `target_dist` inside `current_segment`, computes `ratio = (target_dist - seg_start) / seg_len`, clamps it to `[0, 1]`, and returns `p0 + ratio * (p1 - p0)`. A `seg_len <= 1e-6` guard avoids division by zero. It provides evenly distance-driven targets even when source waypoints are unevenly spaced.

`_desired_speed(s_val, dist_left, v_max)`

This private helper creates a preferred Cartesian speed before joint constraints:

$$v_{bell} = v_{max} 16s^2(1-s)^2.$$

The polynomial is zero at the endpoints and equals v_{max} at $s=0.5$. It is capped by the constant-deceleration stopping speed:

$$v_{\text{brake}} = \sqrt{2 \text{SAFE_DECEL_RATE} \text{dist_left}},$$

from $v^2 = 2ad$. It then applies a nonzero far/near endpoint floor. Floors prevent the loop from making zero progress due to the profile's endpoint zeros, but they are a pragmatic choice that can compete with strict bounds in difficult configurations.

`_joint_limited_speed_bounds(L_vec, dq_curr)`

$L_{\text{vec}} = J^+ \hat{u}$ gives joint velocity per unit Cartesian speed along the current unit path tangent. If commanded Cartesian speed is v , then approximately $dq_i = L_{\text{vec}}[i] * v$.

- Velocity constraint: every nonzero component requires $v \leq \text{MAX_JOINT_VEL} / \text{abs}(L_i)$; their minimum is v_{vlim} .
- Acceleration constraint: the next joint velocity must lie between $dq_{\text{curr}}[i] - \text{MAX_JOINT_ACC} * DT$ and $dq_{\text{curr}}[i] + \text{MAX_JOINT_ACC} * DT$. Division by positive or negative L_i produces a valid speed interval for that joint; all three intervals are intersected.
- If the intersection is inconsistent, the implementation sets the lower bound equal to the upper bound instead of raising an error.

This function is the key mathematical connection between a smooth visible pen path and safe joint-level motion.

`generate_continuous_trajectory(points, q_start, v_max=MAX_LINEAR_SPEED)`

What it does: Public planner entry point. It returns `[]` for fewer than two points, a one-pose list for a near-zero-length path, otherwise a list of 3-element joint arrays.

Algorithm:

1. Convert input to an array, calculate segment vectors, Euclidean lengths, cumulative distances, and total arc length.
2. Initialise travelled distance, segment index, $q_{\text{curr}}=q_{\text{start}}$, and zero previous joint velocity.
3. At each frame, calculate normalized progress and desired/braking speed.
4. Normalize the current segment vector to $\hat{u}$ (use $[1, 0]$ for a degenerate segment), compute J , J^+ , and L_{vec} , then collect joint-derived bounds.
5. Choose v_{actual} , integrate distance as $v_{\text{actual}} * DT$, and clamp the final step to the exact total distance.
6. Move to the appropriate segment, interpolate a Cartesian target, solve IK with the previous pose as the warm start, estimate $(q_{\text{next}} - q_{\text{curr}}) / DT$, and append a copy of q_{next} .
7. Continue until the whole arc length has been traversed.

The IK success flag is discarded. Consequently, the planner maintains a sequence even when an individual target is infeasible, but that sequence may no longer trace the requested Cartesian path accurately. A production hardening step would collect and expose failed frames.

4. cv.py — image analysis and image-to-workspace conversion

Purpose and project role

This is the raster-to-stroke subsystem. It accepts an OpenCV image and returns a list of mapped physical polylines. `gui.py` invokes its stages separately to show previews, while `Pen_up_down.py` uses its one-call `image_to_robot_paths()` entry point. `Calibration.py` imports it through a wildcard, although its actual image interaction comes from `gui.run_gui()`.

The file separates three visual concepts: clean outline contours, horizontal hatch strokes for dark regions, and the physical-coordinate transformation required by the robot.

AnnotatedPath(list)

This tiny class behaves exactly like a normal list of coordinate tuples but carries an `is_hatch` attribute. `map_paths_to_workspace()` creates it. The richer simulator treats paths as ordinary coordinate sequences; GUI code can inspect metadata to distinguish hatch fill from outlines. This is a lightweight alternative to a separate path data structure.

xdog_filter(image, sigma, k_sigma, epsilon, phi, gamma)

Purpose: Produces a sketch-like grayscale image that emphasizes dark lines and detail.

1. Converts BGR input to grayscale if needed.
2. Applies CLAHE, local contrast enhancement, so details remain visible across uneven lighting.
3. Normalizes intensity to `[0, 1]` and calculates two Gaussian blurs: `g1` at `sigma`, `g2` at `sigma*k_sigma`.
4. Computes Difference of Gaussians: `dog = g1 - gamma*g2`.
5. Leaves values above `epsilon` white; below it uses `1 + tanh(phi*(dog-epsilon))`, a soft threshold whose `phi` controls steepness.
6. Rescales to an 8-bit image.

XDoG is preferable here to a basic edge detector because it creates a controllable line-art interpretation suitable for a pen.

automatic_parameters(image)

This helper estimates XDoG values when any caller passes `None`. It converts to grayscale, uses a 3×3 median filter, and measures mean absolute difference from the original as a noise estimate. It also calculates image scale from the larger dimension. Noise and resolution are linearly transformed then clipped into sensible slider-like ranges for `sigma`, `k_sigma`, `epsilon`, `phi`, and `gamma`. It returns the five values in that order. The configuration flag named `XDOG_AUTO_TUNE` does not gate this function.

remove_small_components(binary_image)

Binary sketches use black (0) foreground and white (255) background. The function forms a foreground mask, runs 8-connected component labelling, and keeps only components whose pixel area exceeds a resolution-dependent threshold: `clip(image_size * 0.00015, 50, 2000)`. It returns a fresh white image with retained components restored in black. This prevents dust/noise becoming expensive, tiny robot strokes.

preprocess_image(raw_image, sigma=None, k_sigma=None, epsilon=None, phi=None, gamma=None)

This is the outline-preprocessing entry point:

1. Convert input to grayscale when necessary.
2. Apply a bilateral filter (`d=9`, `sigmaColor=75`, `sigmaSpace=75`) to suppress noise while preserving edges.

3. Fill any missing XDoG values through `automatic_parameters()`.
4. Run `xdog_filter()`.
5. Use Otsu binary thresholding, which selects a threshold from the output histogram.
6. Remove small connected components and return the cleaned binary sketch.

Both GUIs pass explicit slider values, so their outputs are controlled by the user; API callers may rely on the automatic fallback.

`extract_smoothed_contours(binary_image)`

Purpose: Turn cleaned line pixels into simplified, smooth outline path records.

The function inverts black-on-white data so OpenCV sees foreground, uses `findContours(..., RETR_LIST, CHAIN_APPROX_NONE)`, then processes every contour:

- Remove degenerate/singleton results and paths shorter than `MIN_CONTOUR_ARC_LEN`.
- Remove adjacent repeated pixels and reject paths under `MIN_CONTOUR_POINTS`.
- Apply Ramer–Douglas–Peucker approximation with `APPROX_POLY_EPSILON`; if it degenerates, keep the original contour.
- Determine approximate closure by testing endpoint distance `< 3` pixels.
- Attempt a B-spline fit via `splprep` with order at most three and smoothing `SPLINE_SMOOTHING`; resample at `max(8, int(arc_len*0.4))` points. On any spline exception, use the polygonal approximation.
- Return dictionaries containing `points`, bounding-box `x/y`, and `is_hatch=False`.

The bounding box does not describe a robot constraint; it is later used for stable image-space sorting.

`generate_hatching_paths(binary_image, step=4)`

Purpose: Generates horizontal fill lines through dark regions. A 3×3 dilation changes the mask used for hatching. For each image row sampled every `step` pixels, the function finds contiguous black intervals by differentiating a padded zero/one row. It traverses alternate rows in opposite directions, making a serpentine path that reduces pen-up jumps.

If a candidate continuation is more than `step*3` pixels away, it ends the current hatch path; segments under two pixels are skipped. It then rejects total hatch paths shorter than `step*5`, assigns bounds, and returns records with `is_hatch=True`. In `image_to_robot_paths()`, outline extraction is intentionally performed on an interior-masked image while hatching is formed from the original binary image, reducing duplicated outline/interior drawing.

`map_paths_to_workspace(paths, image_shape)`

This function is the image-to-robot coordinate boundary. It first sorts path records by top-to-bottom then left-to-right bounding-box position. It calculates a uniform scale:

$$scale = \min(TARGET_CANVAS_W/w, TARGET_CANVAS_H/h).$$

For each pixel (`px`, `py`), it centres image coordinates, scales them, and flips `y` because image `y` grows downward while the robot convention grows upward:

$$x_r = CANVAS_CENTER_X + (px - w/2)scale$$

$$y_r = CANVAS_CENTER_Y - (py - h/2)scale.$$

Paths whose endpoints are sufficiently close are explicitly closed. The result is a list of `AnnotatedPath` objects with hatch metadata preserved. This is where a canvas-calibration error should be fixed, not in the IK solver.

`image_to_robot_paths(raw_image, ...)`

This convenience entry point executes preprocessing, creates the dilated hatch mask, whites out hatch regions in a copy used for contour extraction, gets outlines and original-mask hatches, concatenates both record lists, and maps them to physical coordinates. It fixes hatch spacing at `step=4`, unlike `gui.py`, whose hatch-step slider calls `generate_hatching_paths()` itself.

5. `gui.py` — compact operator approval GUI

Purpose and where it is used

`gui.py` provides the hardware runner's user-facing image selection phase. When the user enters `t` in `Calibration.py`, it calls `run_gui()`, which opens this window and returns either mapped paths or `None`. Running this file directly opens the same UI but has no receiving controller, so it simply closes after confirmation.

It uses `cv.py` stages directly rather than `image_to_robot_paths()` because it needs a configurable hatch-step value and a visual preview before final mapping.

XDoGGUI

Constructor: `__init__(master)`

The constructor creates a left control panel and right 800×800 canvas. It stores `raw_image=None` and `cv_paths=None` until a user has loaded and confirmed an image. It creates Tk variable objects for five XDoG values and hatch step, then constructs sliders, an upload button, and a confirmation button. Defaults are local literals matching the current configuration defaults, not imports from `Configurations.py`.

`create_slider(parent, label, var, min_val, max_val, res)`

A UI helper that creates a labelled `tk.Scale`, binds it to a Tk variable, and connects slider changes to `update_preview`. It has no image-processing logic itself; its purpose is to make every parameter immediately observable.

`load_image()`

Opens a file selector accepting JPG/PNG/JPEG/BMP, reads a selected file in grayscale through OpenCV, stores it in `raw_image`, then calls `update_preview()`. If the user cancels, it does nothing. There is no explicit `imread(None)` validation, so a bad/corrupt file can lead to later CV failure.

`update_preview(*args)`

If no image exists, it returns. Otherwise it runs `preprocess_image()` with current slider values, reconstructs the hatch mask, masks hatch interiors from contour extraction, generates contours plus hatch paths, and draws each polyline on a new white preview image. Outline paths use black and hatch paths use gray. Pillow converts the preview to `ImageTk.PhotoImage`, retaining it on `self.tk_image` so Tk does not garbage-collect it; the canvas either creates or updates its image item.

This method previews image-space paths. It does not map them into centimetres or command hardware.

`confirm()`

Repeats the processing steps using current values, then calls `map_paths_to_workspace(all_paths, self.raw_image.shape)` and stores the result in `self.cv_paths`. It calls `master.quit()` whether or not an image existed; thus closing/confirming without input returns `None`.

`run_gui()`

Creates a root window, constructs XDoGGUI, enters Tk's event loop, copies `app.cv_paths`, destroys the root, and returns the copy. The `if __name__ == "__main__"` block invokes this function for standalone preview use.

Significant design detail

Preview computation is duplicated in `update_preview()` and `confirm()`. That guarantees confirmation recomputes from current controls, but centralising it in a shared helper would reduce future drift. The GUI produces no motor commands itself; the title “Send to Robot” means “return paths to its caller.”

6. `Pen_up_down.py` — integrated simulation and Nano pen interface

Purpose and use sites

This module is an orchestration layer: it contains no new CV or IK mathematics, but combines `cv.py`, `Trajectory.py`, and `Kinematics.py` into an interactive simulation. It also supplies `NanoPenController`, which `Calibration.py` creates at module startup through `from Pen_up_down import *`.

The header mentions a Dynamixel controller integration, but `from dynamixel_controller import DynamixelArm` and construction of `self.arm` are commented out; `self.arm` is always `None` in this repository. The simulator can drive the Nano pen if serial connection succeeds, but its arm visualization is sim-only by default.

`NanoPenController`

`__init__(port='/dev/ttyUSB0', baud_rate=9600)`

Attempts to open a PySerial connection with one-second timeout, stores it as `self.ser`, waits two seconds for a Nano reset, and prints connection status. Any exception is caught, logged, and leaves `self.ser=None`, deliberately allowing simulation to continue. The defaults must match the OS device and Arduino firmware.

`pen_up()` and `pen_down()`

When a serial connection exists, `pen_up()` sends byte `b'U'`; `pen_down()` sends `b'D'`. Each catches write errors and reports them without crashing the UI. Comments note alternate 0/1 firmware protocols. These methods contain no feedback or position verification—the Arduino is assumed to interpret the byte and actuate the Z axis.

`close()`

If the port remains open, issues `pen_up()` first, waits 0.5 seconds, and closes serial. It gives both normal UI closure and error paths a safe “lift before disconnect” behavior.

`Planar3DOFSimApp`

Constructor: `__init__(root)`

Initialises the Tk root, an initial bent joint pose $[\pi/4, -\pi/2, \pi/4]$, image/path state, trajectory arrays, frame index, stroke index, and execution flag. It deliberately sets `self.arm=None`; it creates a Nano controller, sets logical pen state to UP, builds controls/plots, and binds Enter to preview updating.

Its parallel arrays are important:

Array	Meaning
<code>traj_q</code>	Planned three-angle pose per frame
<code>traj_dq</code>	Numerical derivative used for plotted joint velocity
<code>traj_draw_flags</code>	Whether a frame should leave visible ink
<code>traj_pen_status</code>	Human/hardware pen state text for that frame

setup_gui()

Builds controls for image upload, five XDoG sliders sourced from `Configurations.py`, preview update, start/reset, and a post-draw timeline slider. It reports independently whether arm and Nano hardware are connected. Since `self.arm` is fixed to `None`, arm status is normally `SIM-ONLY`. Slider range choices constrain user tuning but do not alter the underlying CV formula.

setup_plots()

Creates an equal-aspect task-space arm/path plot plus one velocity plot per joint. It sets x limits using `MAX_REACH`, draws red velocity-limit lines at $\pm \text{MAX_JOINT_VEL}$, creates drawable artist handles, and calls `draw_arm()` once. The arm line intentionally ends at wrist `p2`; the brush line represents `p2`→`p3`.

reset_sim()

Stops execution, clears image/path/trajectory state, resets indices, lifts the Nano pen if needed, removes all preview/trail artists, clears velocity plot data, disables timeline/start controls, updates status, and redraws the current arm. It does not reset `q_current` to the original initial pose, so reset is principally a drawing-state reset.

on_scroll_timeline(val)

When not executing, changes the visible x-range of each velocity graph to `[val, val + REVIEW_WINDOW]`. It enables post-draw inspection without replaying motion. It is intentionally inactive while the animation is running.

upload_image() and update_preview(event=None)

`upload_image()` rejects clicks during execution, opens a PNG/JPG/JPEG picker, calls `reset_sim()`, reads grayscale image, changes status, and previews it. `update_preview()` pulls all slider values, calls `image_to_robot_paths()`, clears old preview artists, plots each valid mapped path in gray, enables Start, and updates status. Unlike `gui.py`, this path uses CV's fixed hatch spacing because `image_to_robot_paths()` owns the hatch call.

start_drawing()

A small gate: if paths exist, disables Start and delegates to `generate_full_trajectory()`. It prevents double-starting from the UI.

generate_full_trajectory(cv_paths)

This is the simulator's high-level sequence builder.

1. It clears old trajectory state and starts from the current joint pose.
2. The nested `smooth_segment()` applies a Savitzky-Golay filter to each joint only when a segment is longer than `SG_WINDOW_LENGTH`; then it clips results to joint limits. It returns unchanged short paths.
3. For each CV stroke, it uses FK to get current tip position and plans a two-point pen-up travel trajectory at `MAX_LINEAR_SPEED * TRAVEL_SPEED_MULT`.
4. It records optional lifting dwell frames, pen-up travel frames, lowering dwell frames, then a pen-down trajectory for the actual path at normal drawing speed. Parallel draw/status arrays remain exactly aligned with `traj_q`.
5. It converts poses to a NumPy array, estimates `traj_dq = gradient(traj_q, DT)`, sets timeline bounds/plot scales, marks execution active, and enters the animation loop.

The dwell implementation uses repeated references to `q_sim` in a list; it does not mutate those pose arrays later, so this is harmless in current code. Smoothing after a planner designed around limits may alter numerical velocity/acceleration, although angle clipping provides a basic safety bound.

draw_arm()

Calls `arm_link_positions(q_current)`, updates the arm and brush artists, requests redraw, and returns pen-tip `p3`. It is the geometry-to-visualisation boundary.

run_animation_loop()

For each scheduled frame, it loads `q_current`, draw flag, and pen status; draws the arm; optionally calls `self.arm.move_to_angles()` if a future arm wrapper is provided; switches the Nano only on a logical pen-state transition; and extends an ink trail only on drawing frames. It updates a sliding velocity window rather than every historic point—an explicit performance fix so long drawings do not become progressively slower. It schedules the next frame after `int(DT*1000)` milliseconds.

When frames end, it changes status to complete, enables timeline review, and lifts the pen. The string tests ("UP", "LIFTING", "DOWN", "LOWERING") are the synchronization contract between planning labels and Nano movement.

on_close()

Closes the Nano safely, closes an optional future arm object, destroys Tk, and closes Matplotlib figures. A launcher must register it as the window close protocol for this cleanup path to run.

7. Calibration.py — hardware boundary, safe execution, and stroke recovery

Purpose and architectural role

This is the real arm controller. It configures a Dynamixel Protocol 2.0 bus, manually establishes a zero reference, converts planned radians into motor ticks, clamps commands to relative tick limits, drives the pen controller, detects an apparent stall/power loss, and persists enough state to continue after interruption.

It imports the continuous planner, shared `MAX_LINEAR_SPEED/FPS`, DLS IK, all names from `Pen_up_down.py` and `cv.py`, and `run_gui` specifically. The wildcard `CV` import is not required by its current explicit logic. `sys.path.append()` adds the repository parent, although Python normally already makes the script directory (BobRoss) importable when executed as shown.

Top-level objects and hardware configuration

At import/startup, it creates `pen_controller = NanoPenController()`, `portHandler = dxl.PortHandler(DEVICENAME)`, and `packetHandler = dxl.PacketHandler(PROTOCOL_VERSION)`. It exits immediately if opening the configured `/dev/ttyUSB1` port or setting `1_000_000` baud fails.

Item	Role
<code>DXL_IDS=[0, 1, 2]</code>	Expected shoulder, elbow, wrist motor IDs.
<code>ADDR_OPERATING_MODE=11, mode 4</code>	Dynamixel extended-position (multi-turn) configuration.
<code>ADDR_TORQUE_ENABLE=64</code>	One-byte torque switch.
<code>ADDR_GOAL_POSITION=116</code>	Four-byte commanded tick location.
<code>ADDR_PRESENT_POSITION=132</code>	Four-byte measured tick location.
<code>JOINT_INVERT</code>	Per-joint sign correction between mathematical angle and mechanical mounting.
<code>REL_LIMIT_1/2</code>	Tick offsets from newly measured zero; later sorted before clamp.
stall constants	Determine check cadence, expected command change, detected actual movement, and rollback overlap.

ArmStallError

A custom exception representing a servo commanded to move but not measurably moving. It lets the broad main-loop exception handler distinguish a suspected power/mechanical fault from a normal keyboard interruption and roll persisted work backward only for stalls.

Dynamixel helper functions

`read_signed_position(dxl_id)`

Reads four bytes from ADDR_PRESENT_POSITION. The SDK exposes it as unsigned; values above 2,147,483,647 are converted to signed two's-complement form by subtracting 4,294,967,296. Extended-position motion can cross zero, so signed interpretation is necessary for comparisons around calibrated offsets.

`write_signed_position(dxl_id, tick)`

Converts an intended signed tick to integer and, if negative, adds 2^{32} before writing four bytes to ADDR_GOAL_POSITION. It is the exact inverse representation boundary used by every hardware move.

`set_torque(enable)` and `set_operating_mode(mode)`

Loop over all configured IDs and write their torque byte or operating-mode byte. The startup sequence disables torque before changing mode, then enables it only after zero/limit setup. The quit/error paths disable torque.

`check_for_stall(prev_actual, prev_commanded, curr_commanded)`

Reads current ticks for every motor. For each joint, it compares magnitude of commanded change and measured change across the check window. If command delta exceeds STALL_COMMAND_THRESHOLD while actual delta is below STALL_POSITION_THRESHOLD, it raises ArmStallError; otherwise it returns the latest measured positions for the next comparison. This detects a coarse disagreement, not a full control-loop tracking error: bad thresholds, slow motion, or external holding can produce false positives/negatives.

Recovery-log functions

LOG_DIR and LOG_FILE point to BobRoss/logs/current_drawing.json.

- `save_new_session(zero_ticks_list, cv_paths)` creates the directory, converts every coordinate to JSON-safe float pairs and zeros to integers, and writes zero_ticks, strokes, and last_completed_stroke=-1. It runs before drawing begins.
- `update_progress(stroke_index)` reads JSON, replaces completion index, and overwrites it after each whole stroke.
- `load_session()` simply reads/parses and returns JSON.
- `rollback_progress(num_strokes)` handles missing file quietly, otherwise subtracts an overlap count without going below -1, rewrites the file, and reports the revised index. It assumes the most recently recorded strokes may have been partially completed before fault detection.

The log is stroke-granular, not frame-granular. It does not validate malformed JSON; `r` catches FileNotFoundError but not JSONDecodeError.

Top-level calibration phase

After opening the port, startup disables torque, sets extended-position mode, asks the operator to manually align the entire arm along the positive X axis, and reads three current positions as zero_ticks. It then adds each pair of relative limits to that measured zero, stores them in max_ticks/min_ticks (the variable names do not guarantee ordering), waits, and enables torque. Later clamping explicitly takes `min()/max()`, so swapped relative directions still clamp correctly.

This physical zero is vital: mathematical $q=[0,0,0]$ only maps to the real straight-X pose when the operator's alignment and inversion/tick assumptions are correct.

Motion helper functions

radians_to_ticks(radians, zero_offset, invert=False)

Uses the Dynamixel conversion $4096 / (2\pi)$ ticks per revolution. It multiplies by -1 when mechanical direction is inverted and adds the measured zero offset. The result is a desired absolute extended-position tick.

go_to_xy_3dof(target_x, target_y, current_angles)

Calls DLS IK, warns when it does not converge, converts each target angle to ticks, clamps it within that joint's calibrated absolute tick range, prints any safety clamp, writes all goals, prints the target/angles, and returns the target pose for a warm start. It is invoked for manual g movements and prior to each stroke. It sends goals but does not wait for arrival/verify tracking before returning.

follow_path(path_points, current_angles)

Calls `generate_continuous_trajectory()` at configured drawing speed. If no frames result, it returns unchanged angles. Otherwise it reads initial actual ticks, then for every planned pose converts/clamps/writes all joints, sleeps one frame, and every `STALL_CHECK_INTERVAL` frames compares actual versus commanded motion. It returns the final planned pose, which becomes the next stroke's IK start pose.

Interactive command loop

`current_joint_angles` begins at mathematical zero. The loop accepts:

Command	Implementation
q	Disable torque and exit.
z	Command all motors to stored zero ticks and reset mathematical pose.
g	Parse numeric centimetre target and call <code>go_to_xy_3dof()</code> .
t	Open <code>run_gui()</code> , save a session, then for every path: pen up → move to first point → pen down → <code>follow_path()</code> → update stroke index.
r	Load stored zero/strokes/index, rebuild relative absolute limits, return to zero, and execute remaining strokes with the same pen/move/follow/update sequence.

The outer handler catches both `KeyboardInterrupt` and every `Exception`. It raises the pen, rolls progress back only for `ArmStallError`, returns to zero, waits, disables torque, prints the exception, and exits. This favors a conservative stop, but a programming/data error is handled like an interrupted drawing and may mask a traceback during development.

Most important operational constraints

- Verify device paths, baud rate, IDs, motor model control-table addresses, `JOINT_INVERT`, link geometry, canvas mapping, and `REL_LIMIT_*` before enabling torque.
- The controller assumes the Nano command bytes and physical pen mechanics are correct; a serial connection is optional but a failed pen lift can damage artwork.
- The controller's software limits do not replace an emergency stop or careful mechanical limit verification.
- Recovery should be used only if arm/paper/zero relationship remains trustworthy. The saved paths and zero ticks are intentionally reused, but the program cannot detect paper movement or a changed mechanical setup.

Cross-file conceptual summary

The project is best understood as four transformations:

1. **Vision:** `cv.py` converts pixels into geometrically simplified polylines and maps them to centimetres.

2. **Geometry:** `Kinematics.py` maps between centimetres and angles through FK, Jacobian, and DLS IK.
3. **Time parameterisation:** `Trajectory.py` maps a geometric polyline to frames with a speed profile and joint-derived limits.
4. **Embodiment:** `Pen_up_down.py` visualizes those frames and controls an optional pen; `Calibration.py` calibrates their relation to real motor encoder ticks, commands the bus, and records recovery state.

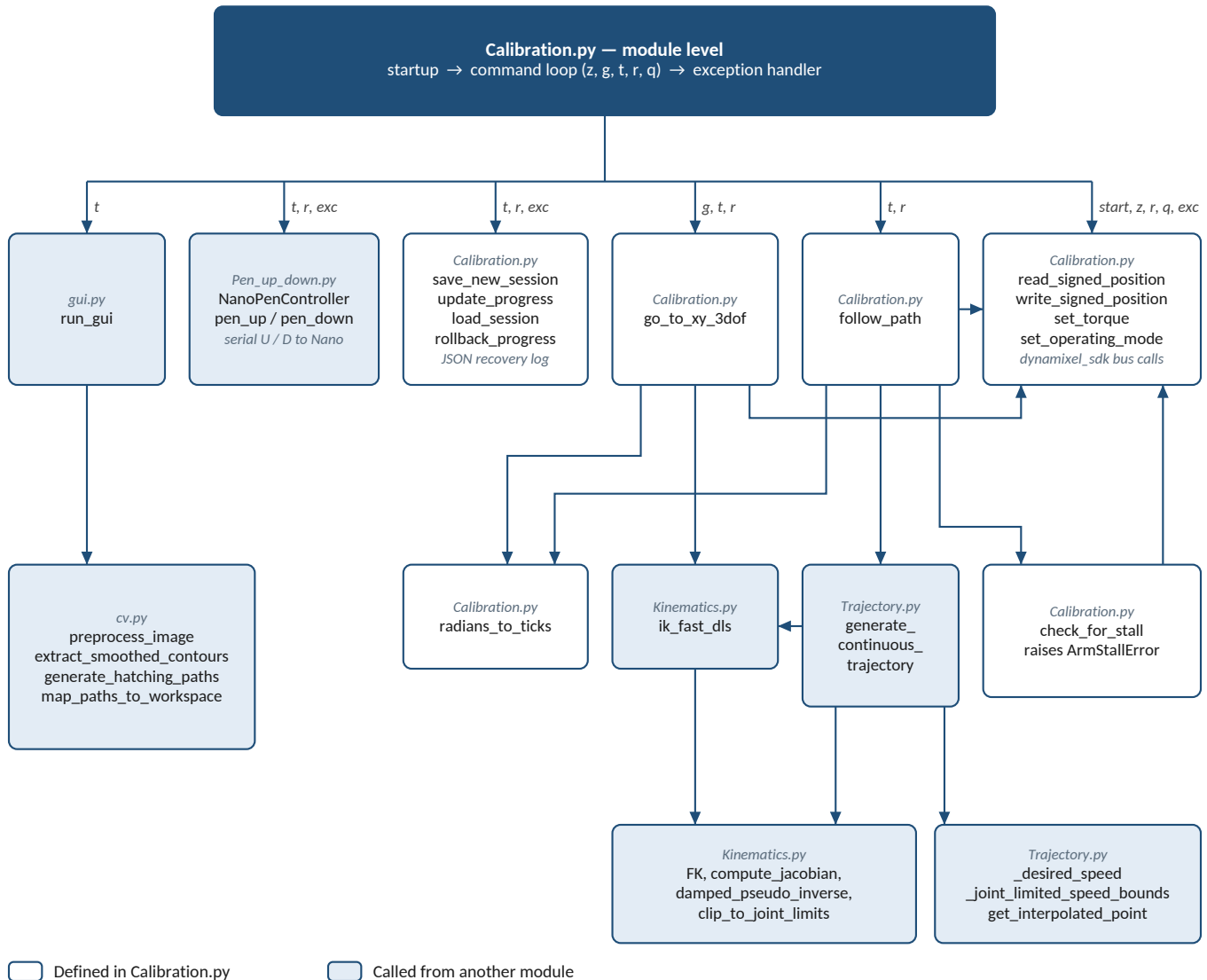
Changing a value or algorithm at one boundary changes assumptions at the next. The safest development order is image preview → simulation → sparse supervised hardware move → simple physical drawing → more complex drawings.

2.2 Architecture Diagrams

The module pipeline diagram appears in Section 2.1 (“Scope”) and the system workflow flowchart in Section 2.3 (“System workflow”); see [Scope](#) and [System workflow](#). The call graph of `Calibration.py` and the circuit diagram of the complete system follow.

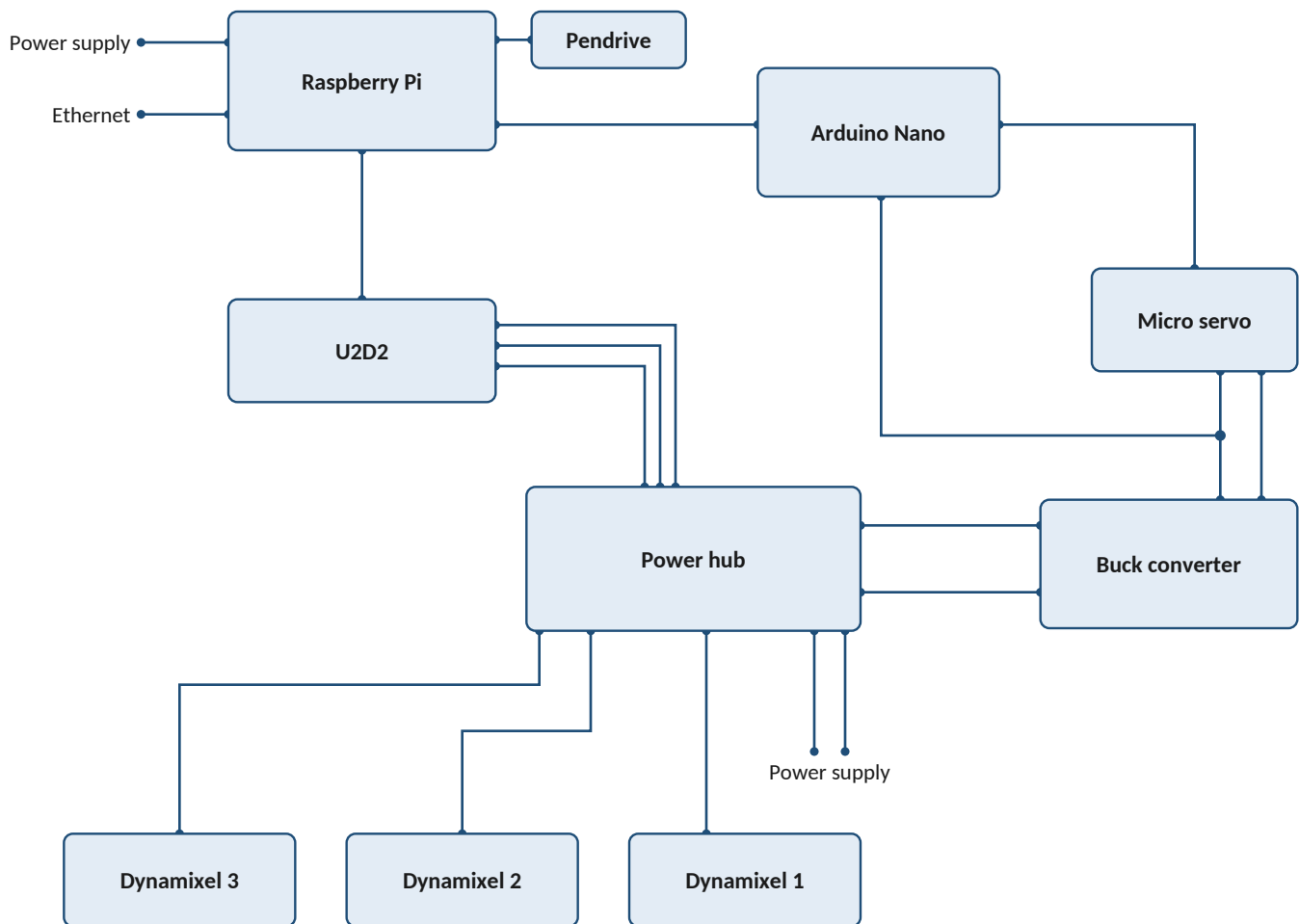
Call graph of Calibration.py

`Calibration.py` is the hardware entry point. Its module-level code performs startup and runs the command loop, which calls the functions defined in the file and the functions of the other modules shown below.



Circuit diagram

The block diagram below shows the electrical connections of the complete system.



Wiring block diagram of the complete system. Parallel lines indicate multi-wire connections.

2.3 README

Bob Ross without ROS

Bob Ross without ROS is an image-to-drawing pipeline for a three-link, planar (3-DoF) robotic arm. It converts an image into outline and hatch strokes, maps them into a physical canvas workspace, solves inverse kinematics (IK), produces joint-limited trajectories, and can command Dynamixel servos. Tkinter/Matplotlib interfaces support previewing and simulating drawings.

The project goal is to reproduce two-dimensional image contours with geometrically accurate paths, smooth joint motion, and practical safety checks for the physical arm.

Safety: `BobRoss/Calibration.py` enables servo torque and can move a real arm. Clear the workspace, verify serial devices, joint directions, and limits, and be ready to remove power. Use simulation and low-risk manual moves before a full draw.

Table of contents

- [Capabilities](#)

- [Repository map](#)
- [System workflow](#)
- [Prerequisites and installation](#)
- [How to run](#)
- [Common use cases](#)
- [Configuration reference](#)
- [Code file reference](#)
- [Debugging and testing](#)
- [Best practices](#)
- [Current limitations](#)

Capabilities

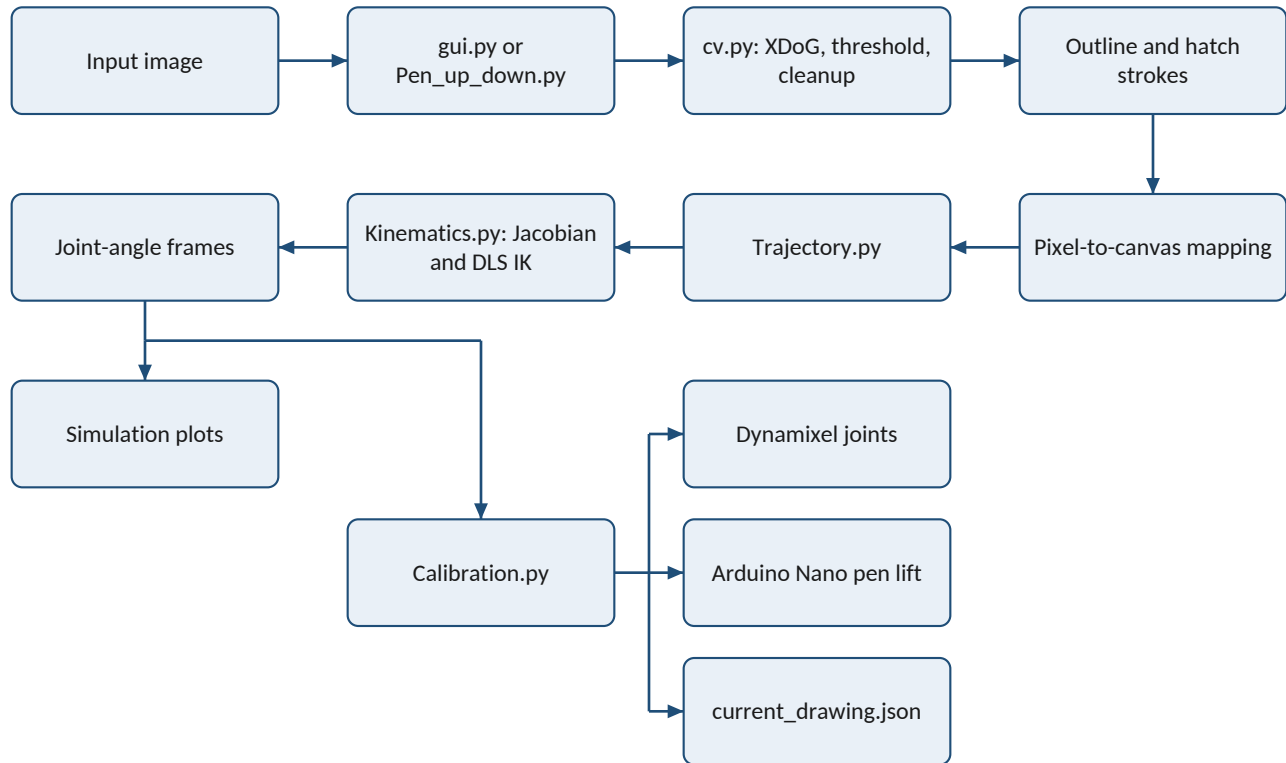
- Converts PNG/JPG/JPEG input (and BMP in the legacy simulator) to drawing paths.
- Uses bilateral filtering, XDoG, Otsu thresholding, component cleanup, contour simplification, B-spline smoothing, and hatch generation.
- Maps image paths to a centimetre-based canvas, solves damped-least-squares IK, and constrains trajectories by configured joint velocity and acceleration limits.
- Displays previews, arm motion, pen state, and joint velocities.
- Supports a serial Arduino Nano pen-lift controller, Dynamixel Protocol 2.0 servo control, stall detection, and stroke-level recovery logging.

Repository map

```
.
├── painting-arm          # Standalone/legacy image-to-simulation program
├── BobRoss/
│   ├── Calibration.py    # Interactive hardware calibration and drawing runner
│   ├── Configurations.py # Shared arm, motion, vision, and canvas constants
│   ├── cv.py             # XDoG image-to-robot-path pipeline
│   ├── gui.py            # Path-selection GUI used by the hardware runner
│   ├── Kinematics.py     # FK, Jacobian, and DLS IK
│   ├── logs
│   │   └── current_drawing.json # Runtime recovery state
│   ├── Pen_up_down.py    # Integrated simulator and Nano pen controller
│   └── Trajectory.py     # Continuous joint-limited trajectory planner
├── CAD Model
│   ├── BoBRoss Assembly.f3z
│   ├── BoBRoss Assembly.step
│   └── Printables
│       ├── base2.stl
│       ├── end effector.stl
│       ├── link1.stl
│       ├── link2p1.stl
│       ├── link2p2.stl
│       ├── link3p1.stl
│       ├── link3p2.stl
│       ├── servo head.stl
│       └── Servo_head_xm430.stl
├── code_summarizer.md
├── Comparison
│   ├── GUI.png
│   ├── Original.png
│   └── Result.png
├── Documentation.pdf
├── GradientDescentBasedIK.pdf
└── README.md
```

BobRoss/ is the integrated vision, planning, simulation, and hardware path. Arm_drawing is a separate, self-contained simulator with different geometry, limits, image processing, and trajectory code; it is not used by the hardware runner.

System workflow



1. A user loads an image in `gui.py` (hardware selection) or a simulator.
2. `cv.py` filters the image with XDoG, binarizes and cleans it, extracts and smooths contours, creates hatch paths, then maps pixels into the configured physical canvas.
3. `gui.py` previews paths. When confirmed, `run_gui()` returns mapped strokes to `Calibration.py`.
4. For every stroke, `Calibration.py` raises the pen, moves to the starting coordinate through IK, lowers the pen, and calls `generate_continuous_trajectory()`.
5. `Trajectory.py` steps along the polyline every DT , uses the Jacobian pseudo-inverse to translate task-space direction to joint motion, and constrains speed by joint velocity and acceleration limits.
6. `Calibration.py` converts radians to servo ticks, clamps ticks around the measured zero pose, writes goals to the three Dynamixels, and checks encoder movement periodically.
7. It saves paths before drawing and advances the completed-stroke index after every stroke. A detected stall rolls the log back three strokes before safe shutdown.

Mode	Entry point	Role	Hardware effect
Path chooser	<code>python BobRoss/gui.py</code>	Tune and preview paths.	None
Integrated simulator	Planar3D0FSimApp launcher below	Modular path/trajectory/velocity simulation.	Nano serial only by default
Hardware runner	<code>python BobRoss/Calibration.py</code>	Calibrates, draws, and recovers.	Dynamixels and optional Nano

Prerequisites and installation

Hardware

Hardware mode assumes a three-link planar arm, three Dynamixel servos addressed as IDs 0, 1, and 2, a Dynamixel-compatible bus adapter, and optionally an Arduino Nano pen-lift controller. The code labels the joints shoulder (XM540), elbow (XM430), and wrist (XM430); verify actual models and IDs.

Current Linux defaults are `/dev/ttyUSB1` at `1_000_000` baud for Dynamixels and `/dev/ttyUSB0` at `9_600` baud for the Nano. Change these for the actual host (for example, COMx on Windows). The Nano firmware must interpret byte U as pen-up and D as pen-down unless the code is changed.

Software

- Python 3+ recommended, pip, and a desktop session with Tk support.
- For Linux hardware mode, serial-device access permissions.
- No `requirements.txt`, `lockfile`, `.env`, environment variables, scripts, or automated installer currently exists.

```
python -m venv Bob
source Bob/bin/activate          # Windows PowerShell: Bob\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install numpy scipy opencv-python matplotlib Pillow pyserial dynamixel-sdk
```

`tkinter`, `json`, `math`, `os`, `sys`, and `time` are standard-library modules; some Linux distributions package Tk separately (for example, `python3-tk`).

How to run

Run from repository root unless noted. Executing a `BobRoss` script adds its containing directory to Python's import path, allowing sibling imports. Connect Ethernet cable between laptop and Raspberry Pi. Ensure IPv4 settings for wired connection is set to Link-Local Only in laptop

Initialising Workflow

```
ssh -X rocket@169.254.225.250
```

Once the dependencies are installed perform this specific set of commands to make it ready for ssh on Raspi as setup already done in it

Setting Up the Environment

```
tmux new -s Bob          # tmux attach -t Bob (if the session is already made)
```

Try to use and make a tmux session just to be safe so that buffer of raspi doesn't overflow and unexpected stopping of program does not occur.

Load the dependencies

For running code perfectly without any error we should use the custom bash file made for it to automatically make everything up for final file to run.

```
source BobRoss.sh
```

Calibrate and draw with hardware

Review serial ports, IDs, inversion flags, relative tick limits, physical link lengths, and canvas mapping first. Send a picture to the Raspberry PI with the following command in a new terminal

```
scp Pictures/picture.jpeg rocket@169.254.225.250:/home/rocket/Desktop/B0B/Bob_Ross/BobRoss/
```

then run in SSH

```
python3 Calibration.py
```

The script opens the Dynamixel port at startup, switches to extended-position control, asks you to align the entire arm straight along the X axis, records zero ticks, calculates relative safety limits, and enables torque. Upload an image, adjust XDoG and **Hatch Step**, and click **Confirm & Send to Robot**. It previews paths, generates travel/draw trajectories, plots velocities, and uses the Nano if available. Its Dynamixel wrapper import/construction is commented out, so it reports ARM: SIM-ONLY by default. It then accepts:

Command	Action
z	Return joints to calibrated zero.
g	Prompt for a target (x, y) in centimetres and move through IK.
t	Open the image GUI, save a new session, and draw each confirmed stroke.
r	Load the recovery log and resume after the saved completed stroke.
q	Disable torque and exit.

Use small, supervised g moves before t. Recovery reuses the stored zero ticks and returns to zero before it continues.

Common use cases

Reduce noisy or overly dense paths

Run `python BobRoss/gui.py`, load the portrait, and adjust the XDoG controls. Raise **Hatch Step** to space fill lines farther apart. For persistent small fragments, adjust the contour/component settings in `Configurations.py`, then retest in a simulator.

Validate the workspace

1. Verify L1, L2, L3, TARGET_CANVAS_W/H, and CANVAS_CENTER_X/Y against the real arm and paper.
2. Calibrate with `Calibration.py`; use g for a few low-risk points.
3. Stop if target, singularity, or safety-clamp warnings appear; correct configuration before drawing.
4. Start with a simple sparse image, not a portrait with dense hatching.

Recover after a stall/interruption

After a stall, the runner lifts the pen, rolls the log back by `ROLLBACK_STROKES_ON_STALL`, returns to zero, disables torque, and exits. Restore power and mechanical safety, rerun `Calibration.py`, calibrate as requested, then use r. Do not recover if the arm, paper, or calibrated reference has moved.

Configuration reference

Shared values — BobRoss/Configurations.py

Group	Key values	Effect
Geometry	L1=15.9, L2=15.05, L3=11.1, L_TUPLE, MAX_REACH	FK, Jacobian, IK, and reachable workspace. Measure physical links before changing.
Limits/timing	JOINT_LIMITS, MAX_LINEAR_SPEED=30.0, MAX_JOINT_VEL=4.0, MAX_JOINT_ACC=1.5, FPS=30, DT	Bounds motion and planner frame time. Conservative values are safer.
IK	IK_MAX_ITER, IK_TOL, IK_DAMPING, IK_MAX_STEP	DLS convergence and stability near singularities.
Trajectory	braking/speed-floor constants, TRAVEL_SPEED_MULT, PAUSE_DURATION	Braking behavior, minimum movement, travel speed, and pen-change pauses.
Filtering	SG_WINDOW_LENGTH=15, SG_POLYORDER=3	Used by the integrated simulator to smooth long joint trajectories.
Vision	XDOG_*, contour thresholds, SPLINE_SMOOTHING, APPROX_POLY_EPSILON	Default vision behavior. XDOG_AUTO_TUNE is defined but not read by current code.
Canvas	TARGET_CANVAS_W/H=23.0, CANVAS_CENTER_X=26.5, CANVAS_CENTER_Y=0.0	Pixel-to-centimetre workspace mapping.

Hardware values

Location	Verify	Reason
Calibration.py	DXL_IDS, BAUDRATE, DEVICENAME, protocol/control-table addresses	Must match the Dynamixel bus.
Calibration.py	JOINT_INVERT, REL_LIMIT_1, REL_LIMIT_2	Physical direction and allowed tick travel around measured zero.
Calibration.py	stall thresholds and rollback count	Balances missed failures against false stall reports.
Pen_up_down.py	Nano port/ baud and U/D bytes	Must match Arduino connection and firmware.
gui.py	Slider defaults/ranges	Local GUI defaults; they do not read Configurations.py.
Arm_drawing	Its own geometry, limits, speeds, and mapping	Only changes the legacy simulator.

Code file reference

Every executable/code configuration artifact is covered here.

Root

painting-arm

BobRoss/ *planning and vision*

BobRoss/Configurations.py

- **Purpose / usage:** Central constants for arm geometry, limits, timing, vision, filtering, and mapping; imported by Kinematics.py, Trajectory.py, cv.py, Pen_up_down.py, and Calibration.py.
- **Necessity:** Keeps the modular simulator, planner, and runner on one model; it does not execute motion.

- **Dependencies / dependents:** Imports NumPy; depended on by the five modules above.
- **Key components:** Link tuple/reach, joint and task-space limits, IK values, trajectory shape values, XDoG/contour settings, and canvas mapping constants.
- **Configuration:** Primary calibration location for physical geometry and canvas coordinates. Shared speed/limit changes affect multiple modules.
- **Debugging:** Wrong placement/reach usually starts here. If changing Savitzky–Golay values, retain a viable window length and polynomial order.

BobRoss/Kinematics.py

- **Purpose / usage:** Planar FK, link positions for rendering, analytical 2x3 Jacobian, damped pseudo-inverse, joint clipping, and iterative DLS IK. Used by Trajectory.py, Pen_up_down.py, and Calibration.py.
- **Necessity:** Converts canvas coordinates into bounded joint poses and exposes the Jacobian needed for trajectory speed limiting.
- **Dependencies / dependents:** Depends on NumPy and shared geometry/IK constants. Depended on by the three modules above; Arm_drawing has its own implementation.
- **Key components:** FK, arm_link_positions, compute_jacobian, damped_pseudo_inverse, clip_to_joint_limits, and ik_fast_dls, which returns (q, success, error_norm).
- **Configuration:** Defaults use L_TUPLE, limits, and IK settings; FK/link/IK functions accept link tuples and IK accepts iteration/tolerance overrides.
- **Debugging:** Check success, error norm, and FK(q) before servo commands. Repeated failure points to unreachable mapping, incompatible limits, singularity, or a poor start pose; tune damping/max step rather than bypassing limit clipping.

BobRoss/Trajectory.py

- **Purpose / usage:** Turns a 2D polyline into one joint-angle vector every DT. Called by Calibration.follow_path() and Planar3DOFSimApp for travel and draw segments.
- **Necessity:** Avoids abrupt waypoint motion by using a shaped Cartesian speed and joint velocity/acceleration constraints.
- **Dependencies / dependents:** Depends on NumPy, configuration motion constants, and kinematics/IK. Depended on by Calibration.py and Pen_up_down.py.
- **Key components:** get_interpolated_point, _desired_speed, _joint_limited_speed_bounds, and generate_continuous_trajectory(points, q_start, v_max).
- **Configuration:** Caller can override v_max; all other limits come from Configurations.py.
- **Debugging:** [] means fewer than two points; nearly zero path length yields one frame. Inspect frame count, v_actual, and dq_curr for slow/erratic paths. Overly aggressive limits may outrun hardware.

BobRoss/cv.py

- **Purpose / usage:** Reusable image-to-path pipeline. gui.py uses its stages for previews; Pen_up_down.py calls image_to_robot_paths(); Calibration.py imports it.
- **Necessity:** Converts raster images to scaled physical polylines, retaining hatch metadata in AnnotatedPath.is_hatch.
- **Dependencies / dependents:** NumPy, OpenCV, SciPy interpolation, and vision/canvas constants; used by gui.py, Pen_up_down.py, and Calibration.py.
- **Key components:** xdog_filter, automatic_parameters, remove_small_components, preprocess_image, extract_smoothed_contours, generate_hatching_paths,

`map_paths_to_workspace`, and `image_to_robot_paths`.

- **Configuration:** Pass XDoG values per image or change shared thresholds/mapping. Hatch spacing is `step=4` pixels by default.
- **Debugging:** Inspect the binary result if output is empty; component cleanup can remove small details. Tune spline/approximation for jagged contours. Correct placement in `map_paths_to_workspace`, not IK.

BobRoss/`gui.py`

- **Purpose / usage:** Compact Tkinter/Pillow image selector. It previews outlines/hatches and returns mapped paths from `run_gui()`; `Calibration.py` launches it for `t`.
- **Necessity:** Provides an operator approval point before a new hardware session is saved and drawn.
- **Dependencies / dependents:** Tkinter, OpenCV, Pillow, NumPy, and `cv.py` path functions. Depended on by `Calibration.py`; it also runs standalone.
- **Key components:** `XDoGGUI.load_image`, `update_preview`, `confirm`, and `run_gui`.
- **Configuration:** Local slider defaults/ranges plus shared physical mapping through `cv.map_paths_to_workspace`.
- **Debugging:** Closing without confirmation leaves `cv_paths=None`. Blank preview can be an unreadable file or aggressive preprocessing. Use a graphical session and check `cv2.imread`/the binary sketch.

BobRoss/ *simulation and hardware*

BobRoss/`Pen_up_down.py`

- **Purpose / usage:** Defines the integrated simulator and Arduino Nano pen serial interface. `Calibration.py` imports `NanoPenController`; users can launch `Planar3D0FSimApp` with the command above.
- **Necessity:** Shows planned motion/velocities and synchronizes pen-state changes to trajectory segments. Nano failure degrades to sim-only operation.
- **Dependencies / dependents:** Tkinter, NumPy, Matplotlib, OpenCV, SciPy signal processing, PySerial, configurations, kinematics, trajectory, and CV modules. Depended on by `Calibration.py`.
- **Key components:** `NanoPenController` opens serial and writes U/D; `Planar3D0FSimApp` previews, plans travel/draw states, smooths eligible segments with Savitzky-Golay, animates, and safely closes connections.
- **Configuration:** Nano path/ baud in the app constructor; trajectory/filter/vision values come from `Configurations.py`. Dynamixel wrapper code is commented out and no wrapper file exists.
- **Debugging:** Serial errors are printed and leave pen sim-only; check device/permissions/firmware. Use `run_animation_loop` to inspect index, pen state, frames, and synchronization. Smoothing is skipped when a segment is not longer than `SG_WINDOW_LENGTH`.

BobRoss/`Calibration.py`

- **Purpose / usage:** Executable real-robot runner: calibrates ticks, configures Dynamixels, converts radians to ticks, draws strokes, detects stalls, and persists/resumes sessions.
- **Necessity:** The hardware bridge with tick clamping and recovery; this is the principal deployment entry point.
- **Dependencies / dependents:** Standard library, NumPy, `dynamixel_sdk`, trajectory/config/kinematics, `Pen_up_down.py`, `cv.py`, and `gui.py`. No file imports it. Treat it as a script: importing opens the port and starts calibration/command loop.
- **Key components:** Signed tick read/write; torque/mode helpers; `go_to_xy_3dof` and `follow_path`; `check_for_stall` and `ArmStallError`; session save/load/update/rollback; commands `z`, `g`, `t`, `r`, `q`.

- **Configuration:** Verify all top-level bus constants, IDs, device path, inversion, relative limits, and stall thresholds. It reads shared `MAX_LINEAR_SPEED` and `FPS`, then derives absolute tick limits from measured zero.
- **Debugging:** Port/ baud failure exits near startup. Wrong direction: stop and correct `JOINT_INVERT`; unexpected clamp: inspect printed limits and `REL_LIMIT_*`. A stall means command changed but encoder motion did not; inspect power, cabling, binding, IDs, and thresholds. The broad handler homes/disables torque—preserve console output and inspect the log.

Runtime and editor files

BobRoss/logs/current_drawing.json

- **Purpose / usage:** Runtime stroke-recovery data. `Calibration.py` overwrites it before a draw, advances it after each stroke, and reads/rolls it back for recovery.
- **Necessity:** It is the persistent state for `r`. The committed file is empty, so recovery is unavailable until a session begins.
- **Dependencies / dependents:** JSON-only; written and consumed only by `Calibration.py`.
- **Key fields / configuration:** Valid data contains `zero_ticks`, `strokes`, and `last_completed_stroke`; do not edit while active.
- **Debugging:** Empty or malformed JSON raises `JSONDecodeError` in the current recovery path. Discard or repair it only while idle and only when intentionally abandoning recovery.

`GradientDescentBasedIK.pdf` is reference documentation rather than code. The active runnable IK is the damped-least-squares implementation in `BobRoss/Kinematics.py`.

Debugging and testing

Safe initial checks

```
# Syntax-check source without running Calibration.py's hardware startup.
python -m py_compile BobRoss/Calibration.py BobRoss/Configurations.py \
  BobRoss/Kinematics.py BobRoss/Pen_up_down.py BobRoss/Trajectory.py \
  BobRoss/cv.py BobRoss/gui.py
```

There is no committed test suite or CI. New tests should target pure functions in `cv.py`, `Kinematics.py`, and `Trajectory.py`; mock Dynamixel/serial calls instead of using live hardware.

Symptom	Likely cause	Resolution
Import error	Package absent from selected interpreter.	Activate <code>.venv</code> , install listed packages, check VS Code interpreter.
TclError / no GUI	Headless machine or missing Tk.	Use a desktop display/backend and install Tk support Or sun command with <code>-X</code> tag to open preview on PC itself(raspi desktop screening).
Port/ baud startup failure	Bad device path, permissions, power, cable, or baud.	Verify enumeration, permissions, power, and <code>DEVICENAME/BAUDRATE</code> .
Pen reports sim-only	Wrong Nano device, firmware, or permissions.	Check Nano port/ baud and U/D protocol.
Arm moves opposite direction	Bad inversion, zero, or mounting assumption.	Remove power, correct <code>JOINT_INVERT</code> , recalibrate, use a small supervised g.
Target warning or bad placement	Mapping, geometry, limits, or singularity.	Validate geometry/canvas values and test individual points.

Symptom	Likely cause	Resolution
Empty/noisy paths	Input or CV settings.	Inspect preview/binary image; tune XDoG, hatch step, component and contour thresholds.
Recovery fails	No valid session or unsafe repositioning.	Start a new session or repair JSON only while idle; resume only with a trusted setup.

Useful breakpoints/log points: `cv.py` (`binary_image`, path counts, mapped paths), `ik_fast_dls` (success/error/FK result), `generate_continuous_trajectory` (frame count, `v_actual`, `dq_curr`), and `Calibration.py` console output/log JSON (zeros, clamps, strokes, stalls).

Best practices

1. Treat link geometry, canvas mapping, joint limits, inversion flags, and tick limits as one calibration set.
2. Develop without hardware first using the legacy or integrated simulator and the path-preview GUI.
3. Begin hardware work with sparse/high-contrast images and small supervised `g` moves.
4. Keep speed and acceleration conservative until physical performance is verified; never rely solely on software limits as an e-stop.
5. Keep the pen lifted during travel/errors and verify the Nano mechanical action.
6. Add numerical/image tests before changing IK or planner tuning; assert that planned points remain reachable and within limits.
7. Do not import `Calibration.py` as a library: import triggers port opening, calibration, and an interactive loop.

Current limitations

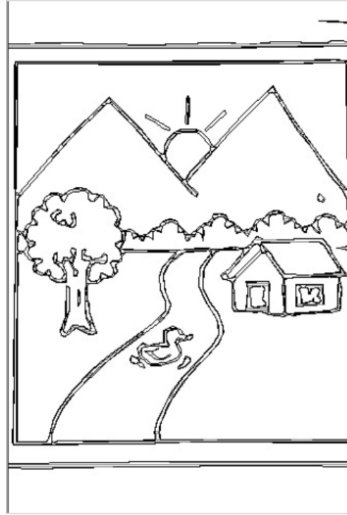
- No package installer, requirements lockfile, automated tests, or CI.
- `Calibration.py` is Linux-device-name specific and executes hardware setup at import time.
- `gui.py` slider defaults are local rather than sourced from `Configurations.py`.

2.4 Comparison Images

The comparison images are stored in the repository folder `Comparison` as `GUI.png`, `Original.png`, and `Result.png`.



Original image



Path preview in the GUI



Result drawn by the arm

2.5 CAD Models

All CAD files for the mechanical design are included in the repository folder `CAD Model`. The folder contains the assembly files (`BoBRoss Assembly.f3z` and `BoBRoss Assembly.step`) and the printable parts as STL files in the `Printables` subfolder.